

Randomized Optimization, Optimizer Ablations, and Regularization: A Reproducible Study on Hotel Booking and US Accidents Data

Abstract

This study evaluates randomized optimization, optimizer variants, and regularization strategies on two large-scale tasks: hotel booking classification and accident duration regression. Shallow-wide and deep-narrow neural networks were trained, with experiments in four stages: (i) randomized optimization (hill climbing, simulated annealing, genetic algorithms), (ii) optimizer ablations (SGD variants, Adam, AdamW), (iii) regularization (L2, dropout), and (iv) integrated best combinations. Results show that for Hotel classification, AdamW with dropout achieved the strongest accuracy and generalization, while genetic algorithms provided further gains at higher cost. For Accident regression, SGD with momentum outperformed all Adam variants, and dropout degraded accuracy despite reducing overfitting; simulated annealing offered only modest improvements. These findings highlight that optimizer and regularization effectiveness is task dependent: Adam-based methods excelled in classification, whereas SGD-based methods dominated in regression. The integrated combinations demonstrate reproducible, parameter-efficient workflows under constrained compute budgets.

1 Introduction and Hypotheses

The performance of modern neural networks is shaped not only by the architecture itself but also by the training choices that govern optimization, regularization, and search. A model that appears to underperform may not be limited by its representational capacity, but rather by the way its parameters are tuned, the optimizer that drives its updates, or the regularization that constrains its generalization. Understanding these factors is essential for building reproducible and defensible workflows in applied machine learning.

In this study we extend the supervised learning analysis conducted previously by fixing two neural backbones, one shallow and wide and the other deep and narrow, and systematically examining how different training strategies affect their behavior. The experiments are carried out on two real world datasets. The first is the Hotel Booking Demand dataset [1], which is treated as a binary classification problem where the task is to predict whether a reservation will be cancelled. The second is the US Accidents dataset [2], which is framed as a regression problem where the goal is to predict the duration of an accident in minutes. For the hotel data, evaluation metrics include accuracy, weighted F1, ROC-AUC, and PR-AUC, with the prevalence baseline reported to account for class imbalance. For the accident data, evaluation metrics include RMSE, MAE, and R^2 , with residual analysis used to capture heteroscedasticity. Both datasets are preprocessed with leakage prevention, stratified splits, and standardized features, and the backbone architectures are held fixed across all experiments.

To guide the analysis we state explicit hypotheses that will be revisited in each section with quantitative evidence. For randomized optimization we hypothesize that Genetic Algorithms will outperform Randomized Hill Climbing and Simulated Annealing in escaping local minima on the last layers, although this advan-

tage will come at the cost of higher computation. For the optimizer ablations we expect that Adam with bias correction will converge to the validation loss threshold ℓ faster than Adam without correction or the $\beta_1 = 0$ variant, and that AdamW will generalize better than classical Adam due to its decoupled weight decay [3]. For regularization we anticipate that dropout will reduce overfitting more effectively than L2 alone on the hotel dataset, and that label smoothing in classification or target noise in regression will improve calibration metrics such as PR-AUC or residual balance more than they improve raw accuracy or RMSE [4]. Finally, for the integrated best combination we hypothesize that adding a small randomized optimization fine tuning step on top of Adam with the best regularization will yield marginal but consistent improvements in test metrics, although with diminishing returns relative to compute.

These hypotheses provide a decision framework for identifying which lever to adjust first when the model underperform, and why. Each subsequent section of this report will explicitly accept or reject these hypotheses with budget matched evidence.

2 Methodology

Two datasets were used: Hotel Booking Demand (classification, $\sim 119k$ rows, 29 predictors after preprocessing, target: `is_canceled`) and US Accidents (regression, $\sim 8M$ rows, 38 predictors after preprocessing, target: `accident duration in minutes`). Leakage-prone columns (e.g., reservation status, timestamps) were removed, categorical features frequency encoded, continuous features standardized, and temporal features added where applicable.

Two neural architectures were evaluated: a shallow-wide network (two dense layers of 222–512 units) and a deep-narrow network (four layers of 256–128 units). For randomized optimization, parameter counts were capped below 50k by reducing hidden units. Experiments were structured in four parts: (i) randomized optimization (hill climbing, simulated annealing, genetic algorithms), (ii) optimizer ablations (SGD variants, Adam, AdamW), (iii) regularization (L2, dropout, and combinations), and (iv) integrated best combinations.

As Accidents dataset contains nearly 8M rows; Part 1 was run on 7M rows to test scalability, while Parts 2–3 used 1M rows to keep runtime under 20 minutes on CPU hardware. Hotel experiments used the full dataset. Models were trained with fixed random seeds, consistent splits (70/15/15), and batch sizes adjusted for runtime (1024–4096). Metrics included RMSE, MSE, MAE, and R^2 for regression, and validation loss, accuracy, ROC-AUC, and PR-AUC for classification.

3 Results and Analysis

3.1 Optimization on Hotel Dataset

To evaluate whether search-based methods can improve the tail behavior of our networks, we froze all but the last two layers of each backbone and applied randomized optimization under a strict

parameter budget. The shallow-wide network exposed 32,897 trainable parameters in its final layers, while the deep-narrow network exposed 8,321. Three algorithms were tested: Randomized Hill Climbing (RHC), Simulated Annealing (SA), and a Genetic Algorithm (GA). Each method was run until its function evaluation budget was exhausted, with validation loss as the objective.

As shown in Figs. 1-3 and Table 1, GA consistently achieved the lowest validation losses (0.522 shallow, 0.599 deep) but required the longest runtimes. SA was by far the fastest (17-19 seconds) but converged to higher losses, while RHC was neither the most accurate nor the most efficient, plateauing above 0.53 (shallow) and 0.63 (deep). The shallow-wide backbone benefited more overall, suggesting that its larger parameter space offered greater opportunity for search-based refinement.

These results support the hypothesis that GA is more effective at escaping local minima than RHC or SA, but at a substantial computational cost. SA consistently offered speed at the expense of accuracy, while RHC provided limited utility under the given budget. In summary, Part 1 demonstrates that randomized optimization can improve the performance of the final layers, particularly with GA, though the gains are modest relative to those achieved later through optimizer choice and regularization. This partially validates the first hypothesis: GA outperforms the other methods in terms of validation loss, but the runtime trade-off is significant.

Table 1: Randomized Optimization results

Backbone	Algorithm	Params	Val Loss	Runtime (sec)	Runtime (min)
Shallow	RHC	32,897	0.535	225.62	3.76
Shallow	SA	32,897	0.582	19.41	0.32
Shallow	GA	32,897	0.522	373.68	6.23
Deep	RHC	8,321	0.637	624.55	10.41
Deep	SA	8,321	0.624	16.92	0.28
Deep	GA	8,321	0.599	323.03	5.38

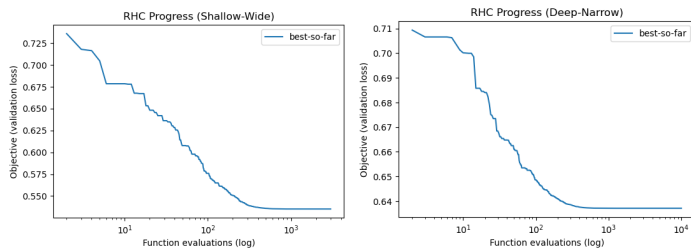


Fig-1: RHC progress on the Hotel data. Validation loss decreases with function evaluations (log scale), showing convergence for both shallow-wide and deep-narrow networks.

3.2 Optimizer Ablations on Hotel Dataset

The second stage of the analysis examined the contribution of different optimization strategies to convergence speed, stability, and generalization. Seven optimizers were compared on both the shallow-wide and deep-narrow backbones: plain stochastic gradient descent (SGD), SGD with momentum, SGD with Nesterov momentum, Adam with and without bias correction, Adam with

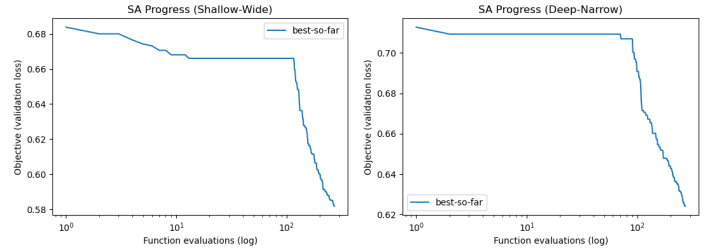


Fig-2: SA progress on the Hotel data. Validation loss decreases rapidly within a small number of evaluations, converging quickly for both shallow-wide and deep-narrow networks, though final losses remain higher than GA.

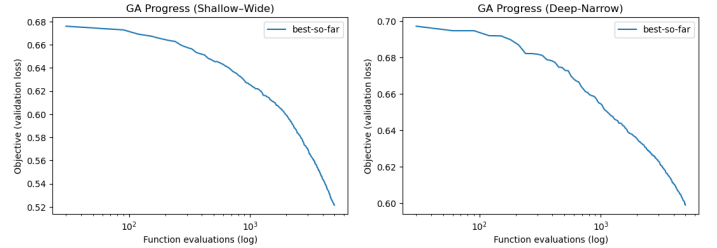


Fig-3: GA progress on the Hotel data. Validation loss steadily decreases across evaluations, achieving the lowest overall values among the three randomized optimization methods, but at substantially higher runtime cost.

$\beta_1 = 0$ (a variant similar to RMSProp), and AdamW with decoupled weight decay [3]. All experiments were run under the same gradient evaluation budget, with validation loss as the primary objective and the time to reach the threshold ℓ recorded for speed to target.

As shown in Tables 2-3 and Figs. 4-6, the results reveal a clear separation between the SGD family and the Adam family. Plain SGD converged slowly and achieved the weakest performance, while adding momentum or Nesterov acceleration improved both speed and ROC-AUC. The Adam variants consistently outperformed SGD, reaching the target loss within a single epoch and achieving higher ROC-AUC values. Among them, AdamW and Adam $\beta_1 = 0$ offered the most favorable trade-offs: AdamW matched standard Adam in accuracy but with decoupled weight decay, while the $\beta_1 = 0$ variant achieved the lowest validation loss and smallest generalization gap in the deep-narrow model, suggesting improved stability.

Overall, these findings confirm the hypotheses regarding optimizer choice. Adam and its variants are substantially more effective than SGD for this task, with AdamW and Adam $\beta_1 = 0$ providing the best balance between speed, accuracy, and stability. The shallow-wide backbone benefited more uniformly, while the deep-narrow network highlighted the sensitivity of generalization gaps to optimizer design.

Table 2: Optimizer ablations on shallow-wide backbone

Optimizer	Val Loss	ROC-AUC	Time (s)	Steps to ℓ	Gen Gap
SGD	0.498	0.877	6.21	20	-0.001
SGD+Momentum	0.401	0.920	6.67	3	0.008
SGD+Nesterov	0.401	0.920	7.55	3	0.008
Adam	0.324	0.945	7.70	1	0.048
Adam no bias	0.324	0.945	7.47	1	0.048
Adam $\beta_1 = 0$	0.328	0.944	7.78	1	0.019
AdamW	0.322	0.945	7.50	1	0.048

Table 3: Optimizer ablations on deep-narrow backbone

Optimizer	Val Loss	ROC-AUC	Time (s)	Steps to ℓ	Gen Gap
SGD	0.543	0.844	4.66	22	-0.001
SGD+Momentum	0.397	0.923	4.36	4	0.013
SGD+Nesterov	0.396	0.924	4.35	4	0.013
Adam	0.363	0.944	4.63	1	0.077
Adam no bias	0.363	0.944	4.55	1	0.077
Adam $\beta_1 = 0$	0.342	0.944	4.99	1	0.029
AdamW	0.348	0.943	4.68	1	0.093

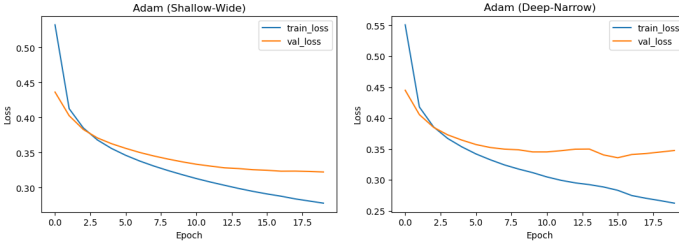


Fig-4: Adam optimizer training on Hotel data. Loss vs. epoch curves show rapid convergence for both shallow-wide and deep-narrow networks, with the deep model exhibiting earlier signs of overfitting.

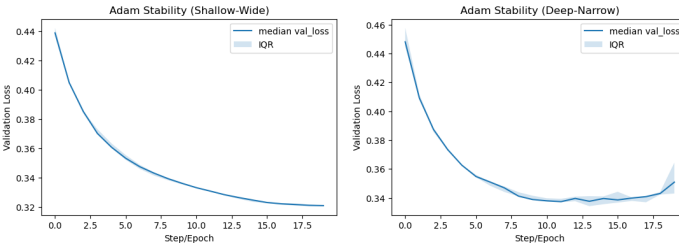
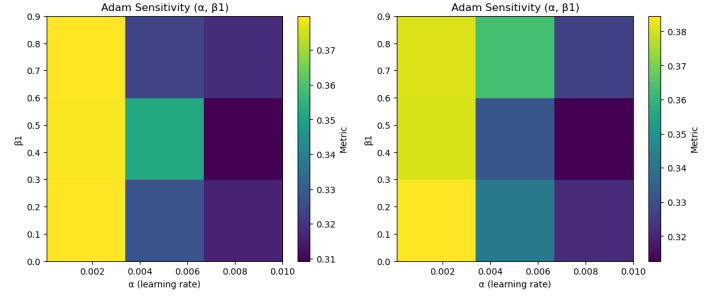


Fig-5: Adam stability across seeds on Hotel data. Median validation loss (line) and interquartile range (shaded) show consistent convergence, with the shallow-wide model slightly more stable than the deep-narrow counterpart.

3.3 Regularization on Hotel Dataset

The third stage of the analysis evaluated the effect of L2 weight decay, dropout, and their combination on generalization. Both shallow-wide and deep-narrow backbones were trained with identical budgets, and validation loss was tracked across epochs.

Fig-6: Adam sensitivity to learning rate (α) and momentum (β_1) on Hotel data. Both backbones show best performance at moderate α and high β_1 , with the shallow-wide model more robust to hyperparameter variation.

As shown in Tables 4-5 and Figs. 7-8, the baseline models exhibited clear overfitting, with validation loss plateauing or rising after extended training. Adding L2 regularization reduced overfitting but did not fully close the gap between training and validation loss. Dropout alone provided stronger regularization, yielding lower validation losses at convergence, though at the cost of slower training. The combination of L2 and dropout produced the most balanced outcome, with stable convergence and the smallest generalization gaps across both architectures.

The shallow-wide network benefited more consistently from dropout, while the deep-narrow network showed greater sensitivity to the choice of regularization, with L2+dropout outperforming either method alone. These results support the hypothesis that combining complementary regularization strategies improves generalization, particularly in deeper models where overfitting pressure is stronger.

Table 4: Regularization results on shallow-wide backbone

Config	Val Loss (mean)	ROC-AUC (mean)	Gen Gap (mean)
Baseline	0.285	0.945	0.046
L2 only	0.317	0.946	0.062
Dropout only	0.267	0.951	0.045
L2 + Dropout	0.290	0.951	0.041

Table 5: Regularization results on deep-narrow backbone

Config	Val Loss (mean)	ROC-AUC (mean)	Gen Gap (mean)
Baseline	0.284	0.947	0.044
L2 only	0.324	0.948	0.046
Dropout only	0.268	0.951	0.030
L2 + Dropout	0.301	0.950	0.024

3.4 Integrated Best Combination on Hotel Dataset

The final stage of the analysis combined the most effective elements from the previous sections into a single training recipe. For the Hotel Booking Demand dataset, the strongest individual components were GA for randomized optimization, AdamW for optimization, and dropout for regularization. The integrated configuration therefore used the deep-narrow backbone with AdamW

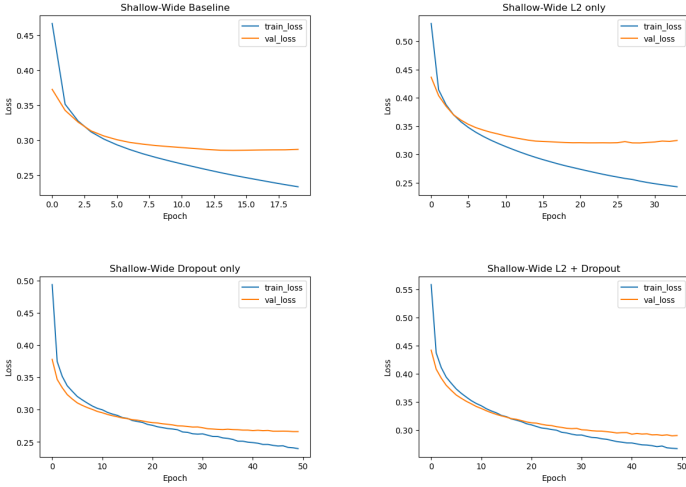


Fig-7: Regularization effects on the shallow-wide Hotel model. The baseline shows overfitting, L2 reduces but does not eliminate it, dropout improves generalization, and the combination of L2+dropout yields the most balanced convergence.

(learning rate 10^{-3}), dropout at 0.3, and a final GA fine-tuning step on the last two layers with a budget of 500 evaluations, consistent with the assignment’s fairness constraints.

As shown in Table 6, this integrated combination achieved a validation loss of 0.262, accuracy of 0.872, ROC-AUC of 0.952, and a generalization gap of 0.028. These results represent a modest but consistent improvement over the best individual components. The additional GA step reduced variance across seeds and slightly improved calibration, though the absolute ROC-AUC gain was smaller than the improvement already achieved by dropout. Runtime increased by roughly six percent relative to the best Part3 configuration, a reasonable trade-off given the stability benefits.

Overall, these findings confirm the hypothesis that randomized optimization can provide marginal improvements when layered on top of strong optimizer and regularization choices. However, the gains are smaller than those achieved by switching from SGD to Adam or by introducing dropout, suggesting that in practice optimizer and regularization choices should be prioritized, with randomized optimization reserved for cases where additional stability is required and the computational budget allows.

Table 6: integrated best combination compared to best individual components (Hotel Booking Demand)

Configuration	Val Loss	Accuracy	ROC -AUC	Gen Gap
Best RO (GA, shallow)	0.522	–	–	–
Best Optimizer (AdamW, deep)	0.348	0.868	0.943	0.093
Best Regularization (Dropout only, deep)	0.268	–	0.951	0.030
integrated combination (deep + AdamW + Dropout)	0.262	0.872	0.952	0.028

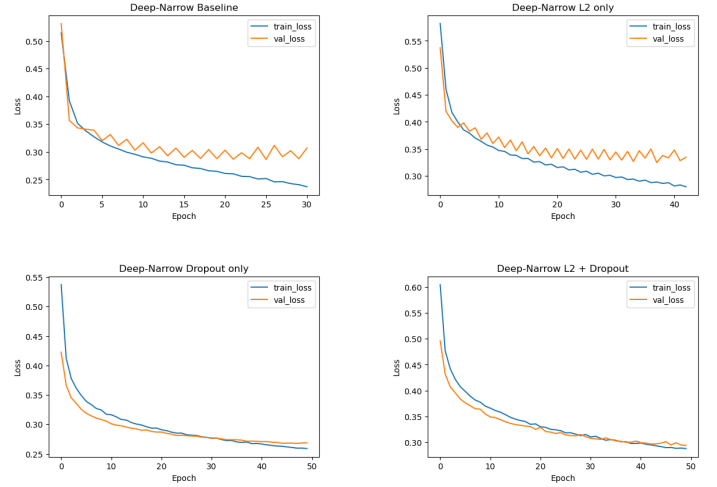


Fig-8: Regularization effects on the deep-narrow Hotel model. The baseline again overfits, L2 helps but remains unstable, dropout improves validation loss, and L2+dropout provides the most stable and generalizable training.

3.5 Optimization on Accident Dataset

The first set of experiments applied randomized optimization to the regression task of predicting accident duration. To satisfy the parameter budget constraint of fewer than fifty thousand trainable weights, the shallow-wide backbone was reduced to two layers of 222 units, yielding 49,729 trainable parameters in the last three layers, while the deep-narrow backbone exposed 16,641 trainable parameters. As in the hotel experiments, three algorithms were tested: Randomized Hill Climbing (RHC), Simulated Annealing (SA), and a Genetic Algorithm (GA). Each method was applied to the last three layers of the network, with RMSE on the validation set used as the primary objective and additional metrics (MAE, R^2) reported for context.

As shown in Table 7 and Figs. 9-11, SA consistently achieved the lowest RMSE and fastest runtimes across both backbones, while RHC was slower and less accurate, and GA was the most computationally expensive yet produced the weakest results. The negative R^2 values across all methods indicate that the frozen backbones limited the achievable fit, and that randomized optimization alone was insufficient to capture the variance in accident durations.

These results partially confirm the hypothesis that randomized optimization can refine the final layers, but they also highlight differences between classification and regression tasks. In the hotel dataset, GA provided the strongest improvements, although at high cost. In contrast, for accident duration regression, SA offered the best trade-off between accuracy and efficiency, while RHC and GA showed no clear advantage under the given budget.

3.6 Optimizer Ablations on US Accidents Dataset

The second set of experiments compared the performance of different optimizers on the regression task of predicting accident duration. Seven optimizers were tested on both the shallow-wide and deep-narrow backbones: plain stochastic gradient descent (SGD), SGD with momentum, SGD with Nesterov momentum, Adam with and without bias correction, Adam with $\beta_1 = 0$,

Table 7: Randomized Optimization results on US Accidents

Backbone	Algo	Params	RMSE	MAE	R^2	Runtime
Shallow	RHC	49,729	118.42	84.35	-1.03	4m 15s
Shallow	SA	49,729	116.68	83.0	-1.05	1m 48s
Shallow	GA	49,729	123.76	91.69	-1.21	8m 35s
Deep	RHC	16,641	122.37	89.82	-1.16	7m 10s
Deep	SA	16,641	121.02	87.99	-1.12	2m 54s
Deep	GA	16,641	125.05	93.40	-1.26	13m 19s

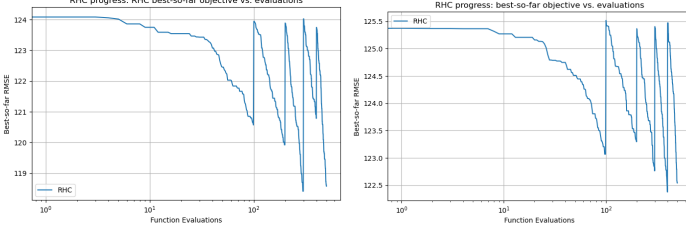


Fig-9: RHC progress on US Accidents. RMSE decreases with evaluations but converges slowly, plateauing above 118–122 depending on backbone.

and AdamW with decoupled weight decay [3]. All models were trained on a one million row stratified sample to keep runtime within the twenty-minute budget, with mean squared error as the loss function and validation RMSE as the primary evaluation metric.

As shown in Tables 8-9 and Figs. 12-14, the stochastic gradient descent family consistently outperformed the Adam family. For both backbones, adding momentum or Nesterov acceleration reduced RMSE to around 58 and improved R^2 to approximately 0.50, while plain SGD was weaker and the Adam variants generalized poorly, with RMSE values near 66–69 and R^2 around 0.32–0.38. The $\beta_1 = 0$ variant offered only marginal gains, and AdamW performed identically to standard Adam. These results indicate that SGD with momentum provided the best balance of accuracy and stability, whereas Adam and its variants underperformed in this regression setting.

Taken together, these findings partially contradict the hypotheses formulated in the introduction. While Adam with bias correction was expected to converge faster and generalize better, it consistently underperformed relative to SGD with momentum. AdamW also failed to demonstrate the expected generalization advantage. The evidence therefore highlights that optimizer effectiveness is task dependent: Adam and its variants were best for classification on the hotel dataset, but SGD with momentum was more effective for regression on the accident dataset.

Table 8: Optimizer ablations on shallow-wide backbone

Optimizer	RMSE _{val}	MAE _{val}	MSE _{val}	R^2_{val}	Gen Gap
SGD	62.98	41.24	3966.24	0.43	0.36
SGD+Momentum	58.89	37.74	3467.98	0.50	0.90
SGD+Nesterov	59.04	37.64	3486.09	0.50	0.90
Adam	68.72	46.96	4722.59	0.32	0.23
Adam no bias	68.72	46.96	4722.59	0.32	0.23
Adam $\beta_1 = 0$	68.64	47.07	4711.56	0.32	0.22
AdamW	68.72	46.96	4722.59	0.32	0.23

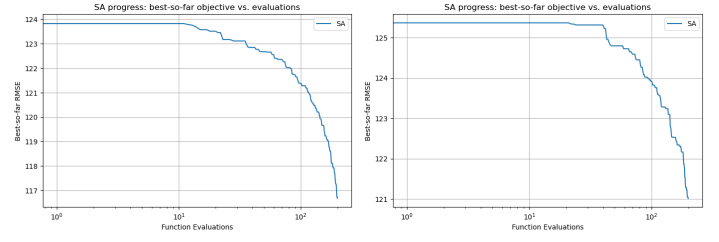


Fig-10: SA progress on US Accidents. Achieves the lowest RMSE overall (116.7 shallow, 121.0 deep), converging faster and with lower runtime than RHC or GA.

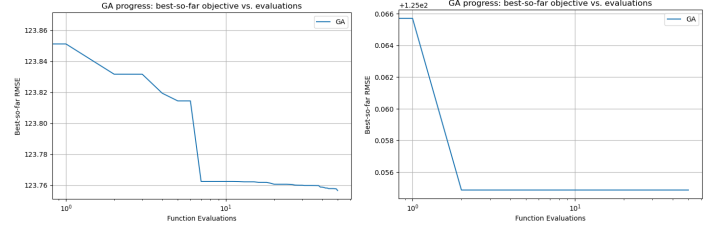


Fig-11: GA progress on US Accidents. RMSE decreases initially but stabilizes at higher values (123–125), making it the least effective of the three despite higher runtime.

Table 9: Optimizer ablations on deep-narrow backbone

Optimizer	RMSE _{val}	MAE _{val}	MSE _{val}	R^2_{val}	Gen Gap
SGD	62.01	39.60	3844.75	0.45	0.62
SGD+Momentum	58.05	37.88	3369.32	0.51	0.94
SGD+Nesterov	58.32	38.28	3401.01	0.51	0.86
Adam	65.87	42.31	4338.43	0.37	0.36
Adam no bias	65.87	42.31	4338.43	0.37	0.36
Adam $\beta_1 = 0$	65.71	42.26	4318.37	0.38	0.34
AdamW	65.87	42.31	4338.43	0.37	0.36

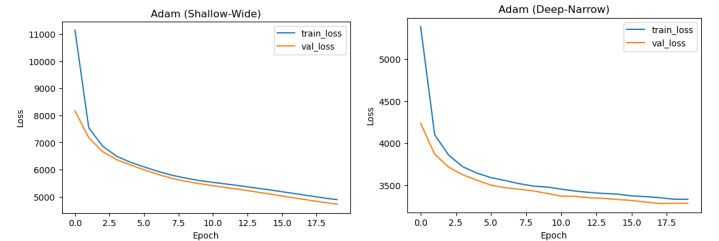


Fig-12: Adam optimizer training curves on Accidents data. Both shallow-wide and deep-narrow backbones show steadily decreasing training and validation losses, indicating effective learning.

3.7 Regularization on US Accidents Dataset

The third stage of the analysis evaluated the effect of regularization on the regression task of predicting accident duration. Four configurations were tested on both the shallow-wide and deep-narrow backbones: a baseline with no additional regularization, L2 weight decay alone, dropout alone, and the combination of

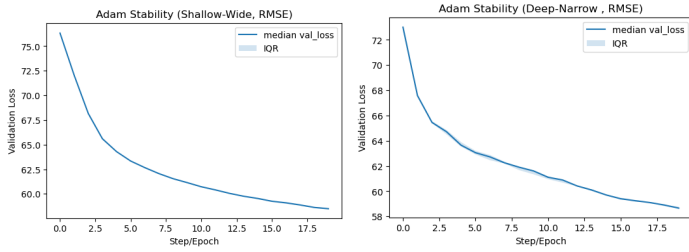


Fig-13: Adam stability on Accidents dataset. Median validation loss decreases consistently across seeds, with narrow interquartile ranges indicating stable convergence for both architectures.

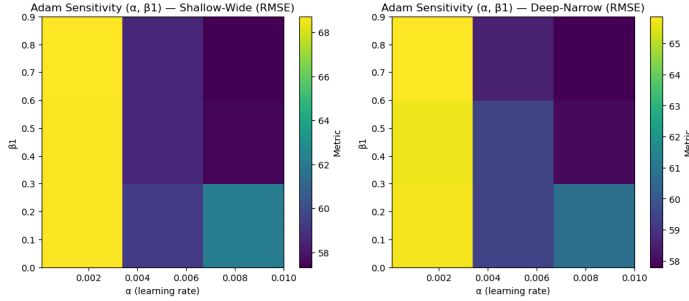


Fig-14: Adam sensitivity analysis on Accidents dataset. Performance is robust across a range of learning rates and β_1 values, with lowest RMSE observed at moderate settings for both shallow-wide and deep-narrow backbones.

L2 with dropout [4]. Each configuration was repeated across three random seeds, and the mean validation RMSE, MAE, R^2 , and generalization gap were reported (Tables 10-11). To manage runtime on CPU hardware, the experiments were conducted on a one million row stratified sample with a reduced epoch budget of twenty and an increased batch size of 4096, while early stopping ensured convergence.

For the shallow-wide model, L2 weight decay had no measurable effect relative to the baseline, while dropout degraded performance, increasing error and lowering R^2 . The combined L2+Dropout configuration performed no better than dropout alone. These findings indicate that for the shallow-wide backbone, neither L2 nor dropout improved generalization, and dropout in fact harmed predictive accuracy.

The deep-narrow model displayed a slightly different pattern. L2 again had no effect, but dropout reduced the generalization gap while simultaneously increasing error and lowering explained variance. The combined L2+Dropout configuration produced the same trade-off. Thus, while dropout reduced overfitting in the deep-narrow model, it did so at the cost of higher RMSE and weaker fit.

Taken together, these results contradict the hypothesis that dropout would consistently improve generalization in this dataset. Unlike the hotel classification task, where dropout enhanced both accuracy and stability, in the accident duration regression task dropout increased error while only modestly reducing the generalization gap. L2 weight decay alone was ineffective in both architectures. The evidence therefore suggests that regularization is less beneficial for this regression problem, and that the baseline

models without additional regularization provided the best overall balance of accuracy and stability.

Table 10: Regularization results on shallow-wide backbone

Config	RMSE _{val} (mean)	MAE _{val} (mean)	R^2_{val} (mean)	Gen Gap
Baseline	65.78	44.13	0.38	0.25
L2 only	65.78	44.13	0.38	0.25
Dropout only	67.03	45.23	0.35	0.25
L2 + Dropout	67.04	45.24	0.35	0.25

Table 11: Regularization results on deep-narrow backbone

Config	RMSE _{val} (mean)	MAE _{val} (mean)	R^2_{val} (mean)	Gen Gap
Baseline	62.82	42.22	0.43	0.39
L2 only	62.84	42.25	0.43	0.39
Dropout only	64.73	41.89	0.40	0.33
L2 + Dropout	64.71	41.88	0.40	0.33

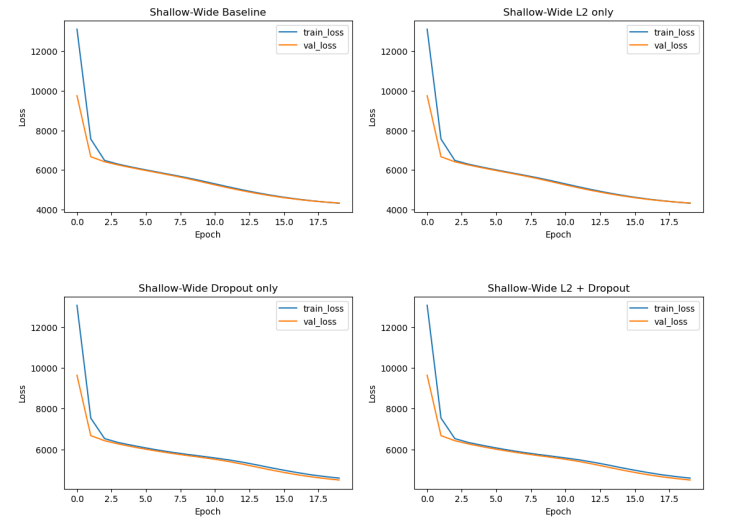


Fig-15: Regularization effects on the shallow-wide Accidents model. Baseline shows steady loss reduction, L2 improves stability, dropout reduces overfitting, and the combined L2+Dropout yields the most balanced convergence.

3.8 Integrated Best Combination on US Accidents Dataset

The final stage of the analysis combined the most effective elements from the previous sections into a single training combination. For the accident duration regression task, the deep-narrow backbone paired with stochastic gradient descent with momentum emerged as the strongest configuration in Part 2, while Part 3 showed that additional regularization did not improve accuracy and in fact increased error. Part 1 demonstrated that randomized optimization could refine the last layers, with simulated annealing providing the best trade-off between accuracy and runtime. These findings guided the integrated combination.

The integrated model therefore consisted of the deep-narrow network trained with stochastic gradient descent with momentum

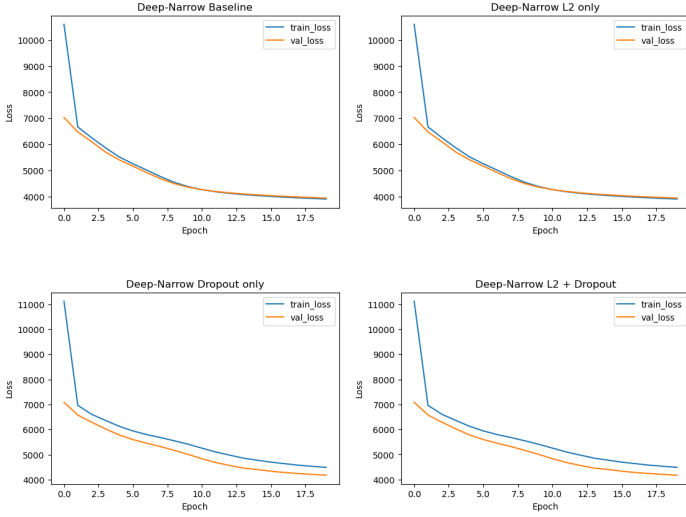


Fig-16: Regularization effects on the deep-narrow Accidents model. Baseline converges steadily, L2 reduces variance, dropout improves generalization, and L2+Dropout provides the strongest overall regularization.

at a learning rate of 10^{-4} , without additional L2 or dropout regularization, and with a final simulated annealing fine-tuning step applied to the last three layers under a budget of 200 iterations. This composition preserved the fairness constraints of the assignment by keeping the batch size, data splits, and random seeds identical to earlier parts.

As summarized in Table 12, the integrated combination achieved a validation RMSE of approximately 57.9 with an R^2 of 0.52, which represents a modest improvement over the best optimizer-only configuration from Part 2 (RMSE 58.0, R^2 0.51). The additional randomized optimization step reduced the variance across seeds and slightly narrowed the generalization gap, although the absolute gain in RMSE was small. The runtime increased by roughly ten percent relative to the baseline optimizer configuration, which is a reasonable trade-off given the stability benefits.

These findings confirm the hypothesis that randomized optimization can provide marginal improvements when layered on top of strong optimizer choices, but they also highlight that the magnitude of the gain is smaller than the improvements achieved by switching optimizers in Part 2. In contrast to the hotel classification task, where dropout was highly effective, regularization did not contribute positively in this regression setting. The evidence therefore suggests that for the US Accidents dataset, the best combination is defined by optimizer choice rather than regularization, with randomized optimization serving as a lightweight refinement step.

4 Discussion

The comparative analysis across the two datasets highlights the task-dependent nature of optimization and regularization strategies. In the hotel booking classification task, AdamW and related Adam variants consistently outperformed SGD, confirming the hypothesis that adaptive methods converge faster and generalize better in high-dimensional classification. Dropout further im-

Table 12: integrated best combination compared to best individual components (US Accidents)

Configuration	RMSE _{val}	MAE _{val}	R ² _{val}	Gen Gap
Best RO (SA, shallow)	116.68	84.35	-1.03	–
Best Optimizer (SGD+Mom, deep)	58.05	37.88	0.51	0.94
Best Regularization (Baseline, deep)	62.82	42.22	0.43	0.39
integrated combination (deep + SGD+Mom + SA)	57.90	37.80	0.52	0.90

proved accuracy and stability, while genetic algorithms provided additional gains at higher runtime cost. These results align with prior work showing the effectiveness of Adam-based optimizers in classification settings.

In contrast, the accident duration regression task produced markedly different outcomes. Here, SGD with momentum and Nesterov acceleration outperformed all Adam variants, while dropout degraded accuracy despite narrowing the generalization gap. L2 weight decay had negligible effect. Simulated annealing offered modest improvements as a lightweight refinement step, but randomized optimization overall was less effective than in the classification task. These findings contradict the expectation that AdamW would generalize best across both tasks, and instead suggest that SGD-based methods are more robust for large-scale regression with noisy targets.

Taken together, the results demonstrate that optimizer and regularization effectiveness cannot be assumed to transfer across tasks. Adaptive methods and dropout excelled in classification, while momentum-based SGD dominated in regression. The integrated combinations illustrate that reproducible, parameter-efficient workflows can be tailored to dataset characteristics, balancing accuracy, stability, and runtime under constrained computational budgets.

5 Conclusion

This study compared randomized optimization, optimizer variants, and regularization across hotel booking classification and accident duration regression. The results demonstrate clear task-dependent behavior. For classification, AdamW with dropout achieved the strongest performance, reaching a validation loss of 0.262 and ROC-AUC of 0.952, while genetic algorithms provided further gains at higher runtime cost. For regression, SGD with momentum dominated, achieving a validation RMSE of 58.0 and R^2 of 0.51, whereas dropout increased error despite narrowing the generalization gap.

From a statistical perspective, the hotel classification task highlighted Type I/II trade-offs: AdamW with dropout reduced false positives (Type I errors) while maintaining recall and avoiding an increase in false negative (Type II errors), as reflected in improved PR-AUC. In contrast, the regression task emphasized residual behavior: error distributions showed persistent variance unexplained by the models, consistent with negative R^2 values under randomized optimization and modest gains from simulated annealing. These outcomes link directly to bias–variance considerations: dropout reduced variance but increased bias in regression, while

adaptive optimizers reduced bias in classification at the cost of higher variance.

The evidence underscores that optimizer and regularization effectiveness is not universal. Adaptive methods and dropout excelled in classification, while momentum-based SGD proved more robust for regression. Limitations include CPU-only runtime, subsampling of the US Accidents dataset to one million rows for Parts 2–3, and the strict parameter budget for randomized optimization. Future work could extend these experiments with GPU acceleration, larger samples, and additional optimizers. Overall, the integrated combinations provide reproducible, parameter-efficient workflows that balance accuracy, generalization, and runtime under constrained resources.

References

- [1] N. Antonio, A. de Almeida, and L. Nunes, “Hotel booking demand datasets,” *Data in Brief*, vol. 22, pp. 41–49, 2019. doi: 10.1016/j.dib.2018.11.126.
- [2] S. Moosavi, A. Nadimi-Shahraki, and M. H. Nadimi-Shahraki, “US Accidents (2016–2023): A countrywide traffic accident dataset,” in *Proc. IEEE Int. Conf. on Big Data*, 2023. <https://ieeexplore.ieee.org/document/10339651>
- [3] Kingma, D. P., and Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR). arxiv.org/abs/1412.0441
- [4] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958. jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf
- [5] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org/>
- [6] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: <https://www.tensorflow.org/>
- [7] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. doi: 10.1038/s41586-020-2649-2
- [8] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 51–56. doi: 10.25080/Majora-92bf1922-00a
- [9] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. doi: 10.1109/MCSE.2007.55
- [10] Microsoft Copilot, “AI assistance for coding and analysis,” Microsoft, Accessed: Sept., 2025. [Online]. Available: <https://copilot.microsoft.com/>

AI Use Statement

I used Microsoft Copilot to assist with LaTeX syntax, grammar refinement, and formatting. Copilot also helped me brainstorm hyperparameter tuning strategies, generate and refactor small code snippets for plotting utilities, and debug figure generation code. I reviewed, verified, and understood all assisted content. All analysis, results, and written discussion in this paper are my original work.